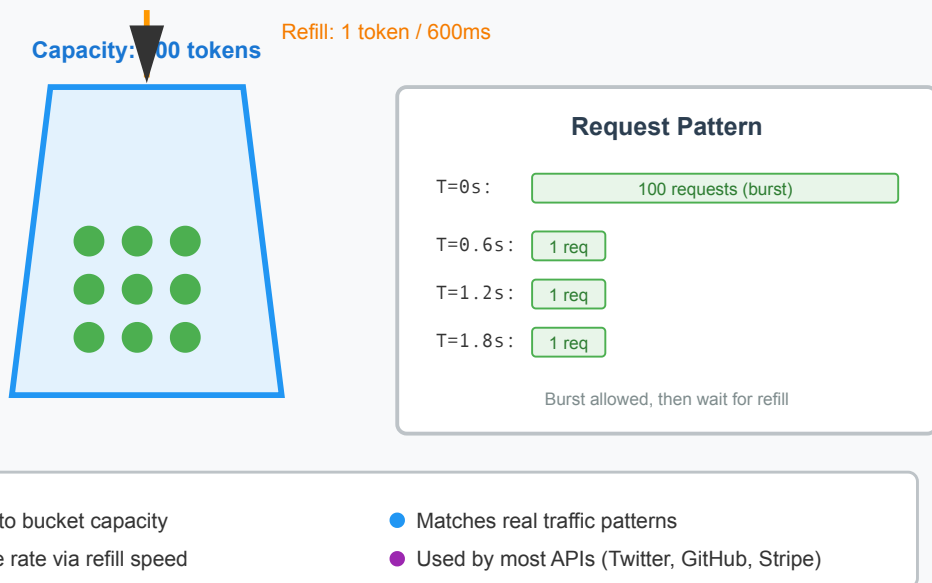


Rate Limiting Patterns

 systemdesignschool.io/fundamentals/rate-limiting-patterns

Token Bucket Algorithm



An API without rate limiting faces abuse. A user script calls the endpoint 100,000 times per second. The database saturates. Response times spike for all users. Legitimate traffic suffers because one bad actor consumes all resources. Rate limiting prevents any single client from overwhelming the system.

Why Rate Limit

Rate limiting protects resources from overuse. APIs impose limits to maintain availability for all clients. Free tier users get 100 requests per minute. Paid tier users get 10,000. This prevents free users from consuming capacity meant for paying customers.

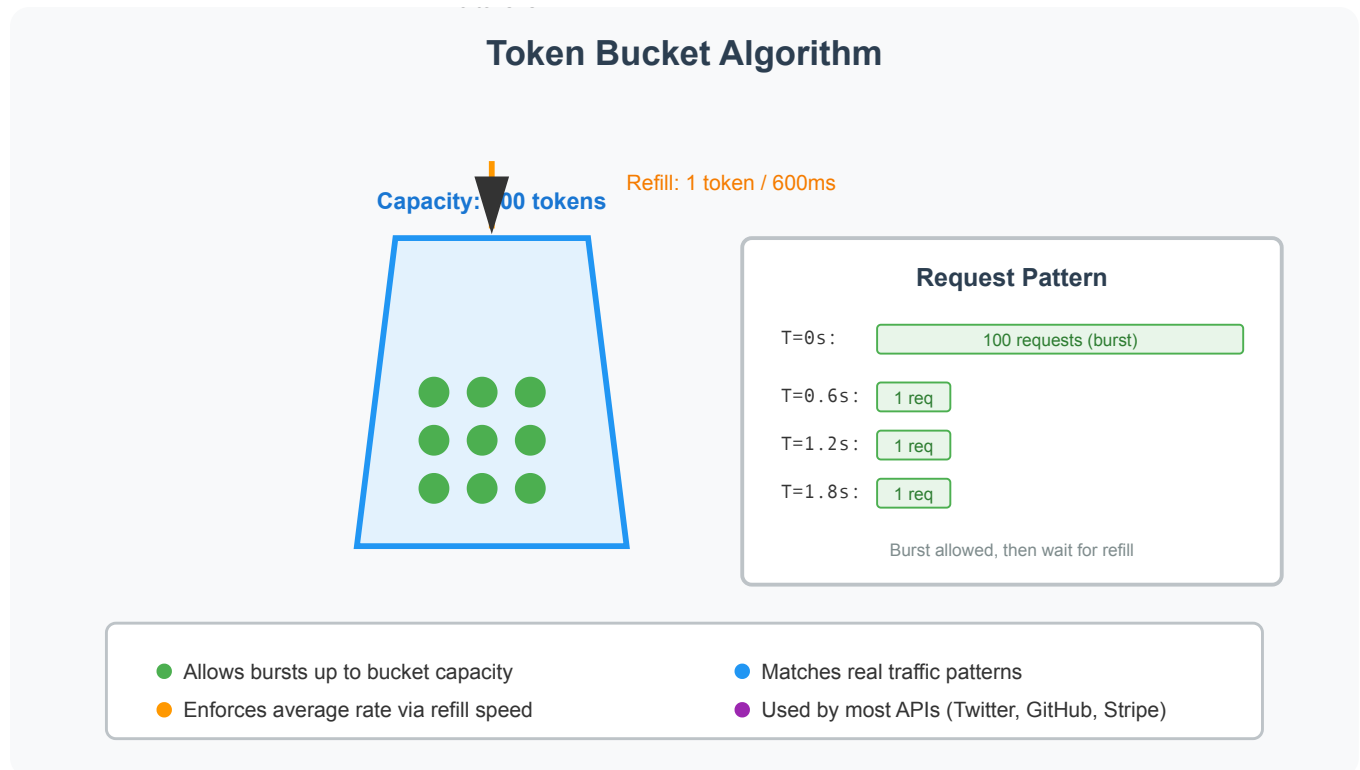
Rate limiting also prevents abuse. Credential stuffing attacks try thousands of password combinations per second. Search crawlers scrape entire websites. Automated bots create fake accounts. Rate limits slow these attacks to manageable levels.

Different resources need different limits. Write operations consume more resources than reads. Creating a post might have a limit of 10 per minute. Reading posts might allow 1,000 per minute. Complex search queries get 100 per minute. Simple lookups get 10,000.

Token Bucket Algorithm

The token bucket holds a fixed number of tokens. Each request consumes one token. Tokens refill at a constant rate. When the bucket empties, requests wait or fail.

A bucket configured for 100 requests per minute holds 100 tokens. Every 600 milliseconds, one token arrives. A client can burst 100 requests instantly, consuming all tokens. Then it must wait 600ms between requests for tokens to refill.



The algorithm uses four parameters. Bucket capacity sets the maximum burst size. Refill rate determines sustained throughput. Current token count tracks available capacity. Last refill timestamp enables rate calculation.

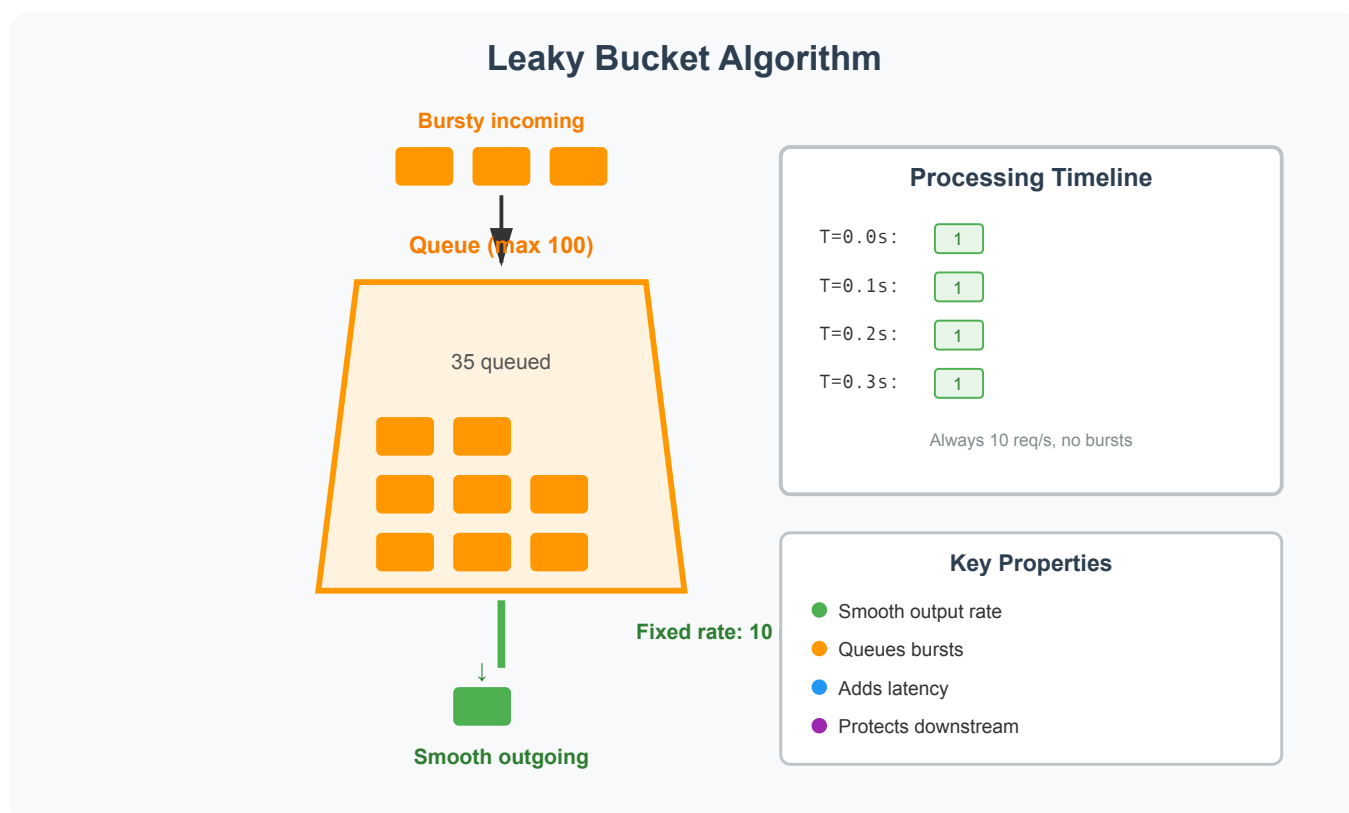
When a request arrives, calculate elapsed time since last refill. Add tokens based on elapsed time and refill rate, capped at bucket capacity. If tokens are available, allow the request and decrement the count. If no tokens remain, reject the request.

The token bucket allows bursts while enforcing average rate. A user can send 100 requests immediately, then waits. This matches real usage patterns where traffic comes in bursts. A dashboard loading makes 20 API calls at once. A background job processes a batch of 50 items.

Leaky Bucket Algorithm

The leaky bucket processes requests at a fixed rate regardless of arrival pattern. Requests enter a queue. The bucket drains at constant speed. When the queue fills, new requests overflow and fail.

Configure a bucket for 10 requests per second with queue size 100. Requests arrive at any rate but process at exactly 10 per second. If 50 requests arrive simultaneously, they queue. The system processes them over 5 seconds at the fixed rate.



The leaky bucket enforces smooth output. Token bucket allows bursts followed by waiting. Leaky bucket spreads requests evenly over time. This protects downstream systems that cannot handle spikes.

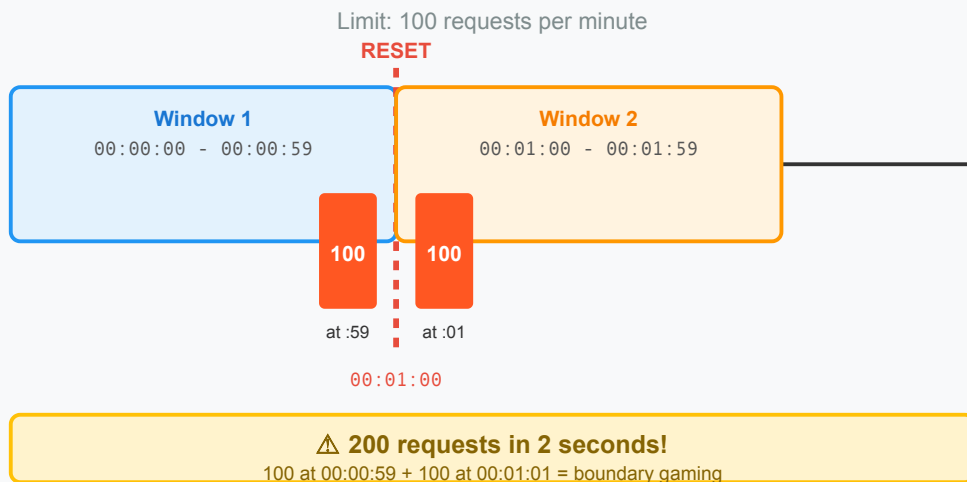
Payment processing APIs use leaky buckets. The payment gateway accepts only 5 transactions per second. Queueing requests ensures compliance with that limit. The queue absorbs bursts during peak traffic while maintaining steady processing.

The tradeoff is latency. Token bucket serves bursts immediately. Leaky bucket queues everything for smooth processing. A user refreshing a page waits while their request sits in queue behind earlier arrivals.

Sliding Window Algorithm

Fixed windows reset at boundaries. A limit of 100 requests per minute resets at 00:00:00. A client sends 100 requests at 00:00:59. The counter resets at 00:01:00. The client sends another 100 at 00:01:01. That's 200 requests in 2 seconds, violating the spirit of the limit.

Fixed Window Problem: Boundary Gaming

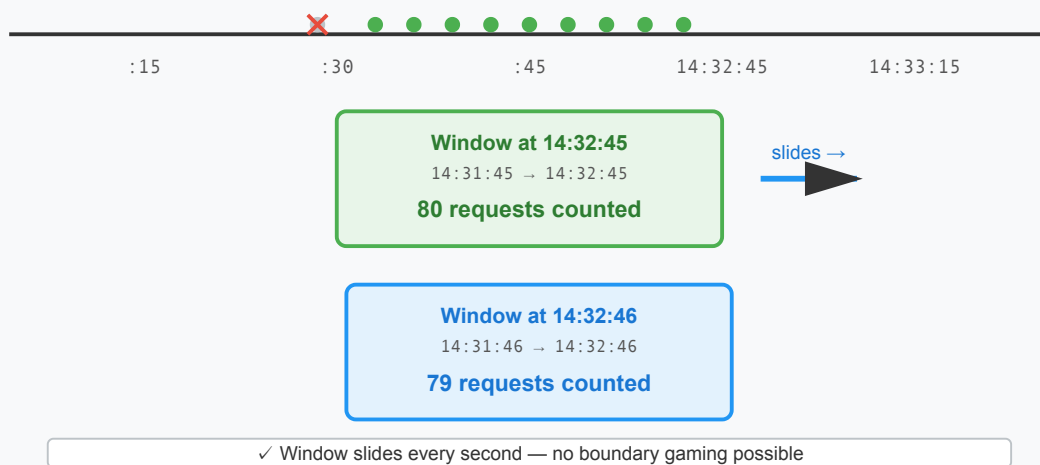


Sliding windows solve this. Track request timestamps for the past minute. When a new request arrives, count requests in the 60-second window ending now. If count exceeds limit, reject.

At 14:32:45, check requests from 14:31:45 to 14:32:45. At 14:32:46, check from 14:31:46 to 14:32:46. The window slides continuously, preventing boundary gaming.

Sliding Window Algorithm

Limit: 100 requests per minute (window slides continuously)



Implementation requires storing timestamps. For a limit of 100 requests per minute, store up to 100 timestamps per user. On each request, remove timestamps older than 60 seconds. Count remaining timestamps. If count is below limit, allow and add new timestamp.

Memory usage grows with limit size. A limit of 10,000 requests per hour requires storing 10,000 timestamps per user. Redis sorted sets work well. Use timestamps as scores and member IDs as values. ZREMRANGEBYSCORE removes old entries. ZCARD counts current entries.

Sliding Window Counter (Approximation)

Storing timestamps is expensive. The sliding window counter approximates using two fixed-window counters with weighted calculation.

Track requests in current minute and previous minute. At 14:32:45, you have 15 seconds into current minute. Weight calculation: previous minute count $\times 0.75$ + current minute count $\times 1.0$. If the sum exceeds limit, reject.

At 14:32:45 with 100 request limit, previous minute had 80 requests, current minute has 30. Weighted sum: $80 \times (45/60) + 30 = 60 + 30 = 90$. Allow the request.

This approximation uses two counters instead of thousands of timestamps. Memory usage drops from $O(\text{limit})$ to $O(1)$. The accuracy trade-off is acceptable for most systems. Errors stay within single-digit percentages.

Distributed Rate Limiting

Single-server rate limiting uses local memory. Distributed systems need coordination across servers. Server 1 and Server 2 each allow 100 requests per minute. A client routes requests evenly across both. They can send 200 requests per minute, violating the intended 100 limit.

Centralized rate limiting uses Redis. All servers check the same counter. Atomic operations prevent race conditions. INCR increments and returns the new value atomically. If the value exceeds limit, reject.

```
def is_allowed(user_id, limit): key = f"rate_limit:{user_id}:{current_minute}"  
count = redis.incr(key) if count == 1: redis.expire(key, 60) # Set expiration on  
first request return count <= limit
```

Redis becomes a single point of failure. Use Redis Cluster or replicas for availability. The network call adds latency but ensures consistent limits.

Local counting with periodic synchronization offers an alternative. Each server maintains local counters and syncs to Redis every second. Server 1 processes 23 requests locally, then adds 23 to Redis. Server 2 adds 19. Redis holds global count 42.

Before allowing a request, check: local count + Redis count $<$ limit. This reduces Redis calls from every request to once per second per server. The tradeoff is potential overages during the sync interval. Two servers might each allow 60 requests before syncing, totaling 120 when limit is 100.

Choosing an Algorithm

Token bucket fits most use cases. It allows bursts matching real traffic patterns while enforcing average rate. APIs use token buckets because client applications legitimately send bursts. Dashboard loads make many calls at once. Background jobs process batches.

Use leaky bucket when downstream systems require smooth traffic. Payment gateways with strict rate limits need steady flow. Video transcoding pipelines process at fixed capacity. Message queues with limited consumer throughput benefit from queueing.

Sliding window provides strictest enforcement. Use it when limit violations have serious consequences. Billing APIs charging money need precise limits. Security-critical endpoints blocking brute force attacks require accuracy. The memory cost is worthwhile when accuracy matters.

Fixed window with sliding window counter offers good balance. The approximation is accurate enough for general protection. Low memory usage scales to millions of users. Most production systems use this approach.

Interview Application

When asked to add rate limiting to an API, start with the resource being protected. Write operations cost more than reads. Complex queries cost more than simple lookups. Different endpoints need different limits.

Explain the algorithm choice. For a Twitter-like API, suggest token bucket. Users post in bursts but average rate stays low. A user might compose 5 tweets in a minute, then nothing for an hour. Token bucket allows the burst while enforcing daily limits.

For distributed rate limiting, mention Redis with atomic operations. Note the latency trade-off. Suggest local counting with periodic sync if latency matters. Explain the potential overage during sync intervals.

Discuss user experience. When a user hits the limit, return HTTP 429 with Retry-After header. Tell them when they can try again. Log rate limit violations to detect abuse patterns. Provide higher limits for authenticated users versus anonymous traffic.

Show you understand the trade-offs between burst allowance, smooth flow, accuracy, and implementation complexity. This demonstrates system design maturity.